Empowerment through quality technical education
AJEENKYA DY PATIL SCHOOL OF ENGINEERING
Dr. D. Y. Patil Knowledge City, Charholi (Bk), Lohegaon, Pune – 412 105
Website: https://adypsoe.in/

LAB MANUAL

OBJECT ORIENTED PROGRAMMING

(VSE-281-AID)

SE (AI&DS) 2024 COURSE

Course Coordinator

Prof. Arvind Gautam

DEPARTMENT OF

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Department of Artificial Intelligence & Data Science

Vision:

Imparting quality education in the field of Artificial Intelligence and Data Science

Mission:

- To include the culture of R and D to meet the future challenges in AI and DS.
- To develop technical skills among students for building intelligent systems to solve problems.
- To develop entrepreneurship skills in various areas among the students.
- To include moral, social and ethical values to make students best citizens of country.

Program Educational Outcomes:

1. To prepare globally competent graduates having strong fundamentals, domain knowledge, updated with modern technology to provide the effective solutions for engineering problems.

2. To prepare the graduates to work as a committed professional with strong professional ethics and values, sense of responsibilities, understanding of legal, safety, health, societal, cultural and environmental issues.

3. To prepare committed and motivated graduates with research attitude, lifelong learning, investigative approach, and multidisciplinary thinking.

4. To prepare the graduates with strong managerial and communication skills to work effectively as individuals as well as in teams.

Program Specific Outcomes:

1. **Professional Skills**- The ability to understand, analyze and develop computer programs in the areas related to algorithms, system software, multimedia, web design, networking, artificial intelligence and data science for efficient design of computer-based systems of varying complexities.
2. **Problem-Solving Skills-** The ability to apply standard practices and strategies in software project development using open-ended programming environments to deliver a quality product for business success.
3. **Successful Career and Entrepreneurship-** The ability to employ modern computer languages, environments and platforms in creating innovative career paths to be an entrepreneur and to have a zest for higher studies.

Table of Contents

Contents

1. Guidelines to manual usage

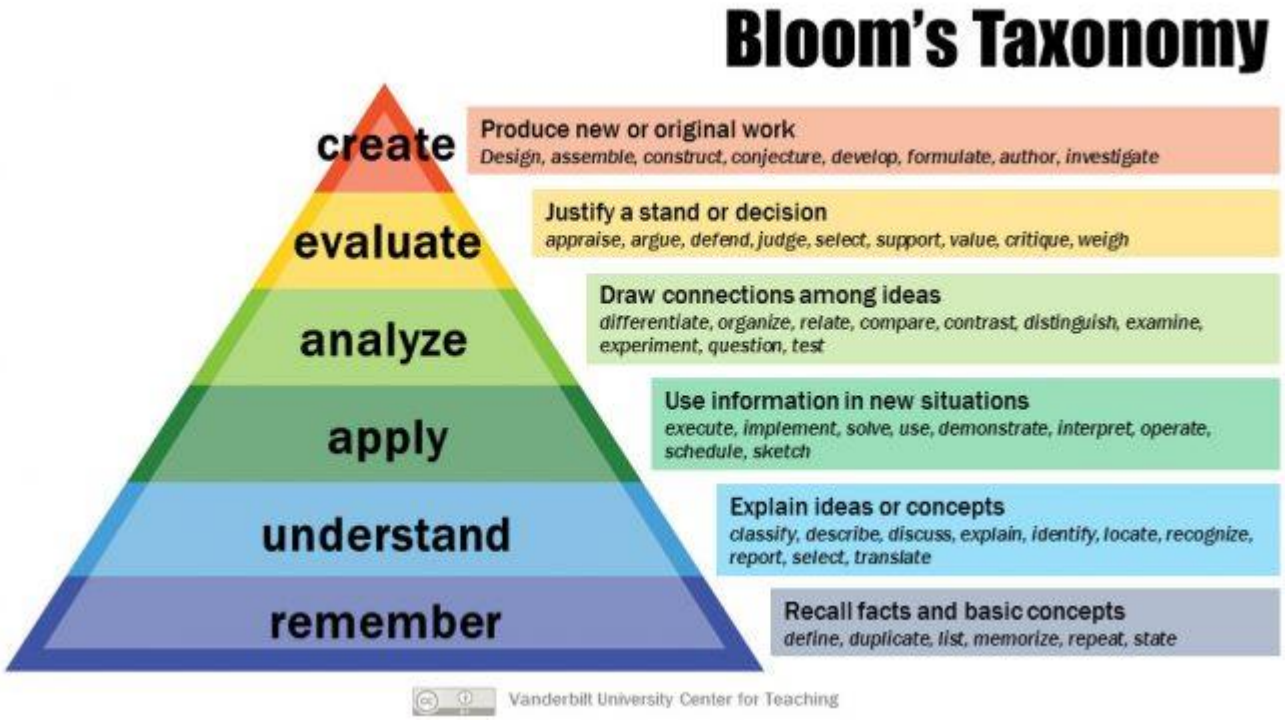
This manual assumes that the facilitators are aware of collaborative learning methodologies.
This manual will provide a tool to facilitate the session on Digital Communication modules in collaborative learning environment.
The facilitator is expected to refer this manual before the session.

Icon of Graduate Attributes

K Applying Knowledge	**A** Problem Analysis	**D** Design & Development	**I** Investigation of problems
M Modern Tool Usage	**E** Engineer & Society	**E** Environment Sustainability	**T** Ethics
T Individual & Team work	**O** Communication	**M** Project Management & Finance	**I** Life-Long Learning

Blooms Taxonomy

Bloom's Taxonomy is a classification of the different objectives and skills that educators set for their students (learning outcomes).



Program Outcomes:

1. **PO1 (Engineering Knowledge):** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization (WK1 to WK4) to develop solutions of complex engineering problems.
2. **PO2 (Problem Analysis):** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3. **PO3 (Design/Development of Solutions):** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs considering public health & safety, whole-life cost, net zero carbon, culture, society and environment. (WK5)
4. **PO4 (Conduct Investigations of Complex Problems):** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis and interpretation of data to provide valid conclusions. (WK8)
5. **PO5 (Engineering Tool Usage):** Create, select and apply appropriate techniques, resources and modern engineering & IT tools (including prediction and modelling), recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6. **PO6 (The Engineer and the World):** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, WK7)
7. **PO7 (Ethics):** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8. **PO8 (Individual and Collaborative Team Work):** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9. **PO9 (Communication):** Communicate effectively and inclusively within engineering community and society, including effective reports, design documentation, and presentations considering cultural, language, and learning differences.
10. **PO10 (Project Management and Finance):** Apply engineering management principles and economic decision-making to one's own work, as a team member/leader, and manage projects in multidisciplinary environments.
11. **PO11 (Life-long Learning):** Recognize the need for and ability for (i) independent and lifelong learning (ii) adaptability to new and emerging technologies and (iii) critical thinking in the broadest context of technological change. (WK8)

Course Name: Object Oriented Programming
Course Code: (VSE-281-AID)

Course Outcomes

1. CO1: Apply fundamental constructs like control statements, for implementing an application.
2. CO2: Implement java programs using, class, objects, constructors in Java, arrays, managing I/O.
3. CO3: Apply object-oriented features like Inheritance, Polymorphism, Dynamic binding for implementing an application.
4. CO4: Apply concepts of exception handling, multi-threading for implementing an application.
5. CO5: Design an interface to connect Java applications with database for performing CRUD operations.
6. CO6: Perform basic statistical analysis and data visualization operations using Java AP

CO to PO Mapping:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	2	1	1	-	-	-	-	-	1	2
CO2	3	3	2	2	1	1	-	-	-	1	2
CO3	3	3	3	2	2	1	1	-	1	1	2
CO4	3	3	2	2	2	2	1	1	1	2	1
CO5	3	3	3	3	2	2	1	1	2	2	2
CO6	3	2	3	2	3	1	2	2	2	3	1

CO to PSO Mapping:

	PSO1	PSO2	PSO3
CO1	3	2	1
CO2	3	3	1
CO3	3	3	2
CO4	2	3	2
CO5	3	3	3
CO6	2	2	3

2. Laboratory Objective

By the end of this laboratory course, students will be able to:

1. To understand and apply **Java programming fundamentals** (loops, conditions, arrays, I/O).
2. To implement **Object-Oriented Programming (OOP)** concepts like class, objects, constructors.
3. To apply **inheritance, polymorphism, abstraction** for reusable application development.
4. To develop robust programs using **exception handling** (try-catch-finally).
5. To implement **multithreading** and synchronization for real-time applications.
6. To connect Java applications with **databases using JDBC** and perform CRUD operations.
7. To perform **basic statistical analysis and data visualization** using Java libraries/JavaFX.
8. To design mini projects (Banking, Inventory etc.) using complete Java application approach.

3. Laboratory Equipment/Software:

Sr. No.	Category	Equipment / Software	Minimum Specification / Requirement
1	Hardware	Computer System (PC/Laptop)	Dual Core / i3 or above
2	Hardware	RAM	4 GB or above (Recommended: 8 GB)
3	Hardware	Storage	50 GB free space (Recommended)
4	Hardware	Network / Internet	Required for JDBC setup, libraries, API simulation
5	Hardware	Printer (Optional)	For printing reports/output
6	Software	Operating System	Windows / Linux / macOS
7	Software	Java Development Kit (JDK)	JDK 8 or above (Recommended: JDK 17/21 LTS)
8	Software	Java IDE	Eclipse / IntelliJ IDEA / NetBeans / VS Code
9	Software	Database Server	MySQL / PostgreSQL
10	Software	JDBC Driver	MySQL Connector/J / PostgreSQL JDBC Driver
11	Software	JavaFX SDK	Required for GUI + interactive charts
12	Software	Apache Commons Math Library	Required for statistical operations
13	Software	CSV File Datasets	Weather / Patient / Student / Employee datasets
14	Software	API Tools (Optional)	Postman / API Mock tools

3.1 Suggested Setup for Each Experiment:

Experiment	Tools Used
Exp 1: Single Inheritance	PC/Laptop, JDK, Java IDE (Eclipse/IntelliJ/NetBeans/VS Code)
Exp 2: Interface & Polymorphism	PC/Laptop, JDK, Java IDE
Exp 3: Abstract Class	PC/Laptop, JDK, Java IDE
Exp 4: ATM Simulation (Exception Handling)	PC/Laptop, JDK, Java IDE
Exp 5: Online Shopping (Exception Handling)	PC/Laptop, JDK, Java IDE
Exp 6: Stock Price Monitoring (Multithreading)	PC/Laptop, JDK, Java IDE, Internet/API simulation (Optional)
Exp 7: Chat System (Multithreading)	PC/Laptop, JDK, Java IDE
Exp 8: Robust Java Calculator	PC/Laptop, JDK, Java IDE
Exp 9: E-commerce Order Processing	PC/Laptop, JDK, Java IDE
Exp 10: Method Overloading (Math Class)	PC/Laptop, JDK, Java IDE
Exp 11: Library Management (Static)	PC/Laptop, JDK, Java IDE
Exp 12: Array Operations	PC/Laptop, JDK, Java IDE
Exp 13: Hotel Room Booking (2D Arrays)	PC/Laptop, JDK, Java IDE
Exp 14: Employee CRUD using JDBC	PC/Laptop, JDK, Java IDE, MySQL/PostgreSQL, JDBC Driver, MySQL Workbench/pgAdmin
Exp 15: Student CRUD using JDBC	PC/Laptop, JDK, Java IDE, MySQL/PostgreSQL, JDBC Driver, MySQL Workbench/pgAdmin
Exp 16: Weather CSV + Stats + JavaFX Charts	PC/Laptop, JDK, Java IDE, JavaFX SDK, Apache Commons Math, CSV Dataset
Exp 17: Patient CSV + Stats + JavaFX Charts	PC/Laptop, JDK, Java IDE, JavaFX SDK, Apache Commons Math, CSV Dataset
Exp 18: Mini Project – Banking System	PC/Laptop, JDK, Java IDE (Optional: File Handling/DB)
Exp 19: Mini Project – Inventory Management	PC/Laptop, JDK, Java IDE (Optional: File Handling/DB)

4. Laboratory Experiment list

Sr. No	Title
List of Assignment (Part A) - Any 4	
Sr. No	**Assignment**
1	Create a Java program demonstrating single inheritance where a subclass extends a superclass and calls its methods.
2	Implement an interface in Java and create multiple classes that implement the interface to demonstrate polymorphism.
3	Write a Java program to create an abstract class with an abstract method and extend it in a subclass that provides its implementation.
4	Develop a Java application that simulates an ATM machine. Implement account balance check, withdrawal, and deposit. Use try, catch, and finally blocks to handle exceptions such as insufficient funds using Arithmetic Exception and invalid input using Illegal Argument Exception. Ensure smooth execution after exception handling.
5	Develop a Java application that simulates an online shopping system. Implement features such as adding items to a cart, calculating total price, and processing payments. Use try, catch, and finally blocks to handle Number Format Exception for invalid input and Arithmetic Exception for calculation errors.
6	Develop a Java application that monitors stock prices in real time using two threads. One thread fetches stock prices from an API and the other displays the prices. Use Thread. sleep to simulate delay and join to ensure both threads complete. Implement thread synchronization to manage shared resources.
7	Create a multithreaded Java application that simulates a basic chat system. Each user thread sends and receives messages. Use is Alive to check thread status and join for synchronization. Implement thread priorities to handle high priority messages and demonstrate thread suspension, resumption, and stopping.
List of Assignment (Part B) - Any 4	
Sr. No	**Assignment**
1	Implement a robust Java calculator program that captures user input dynamically, processes mathematical expressions using conditional logic and looping constructs, and ensures proper error handling.
2	Develop a Java program for an e commerce order processing system where products are initialized using multiple constructors, user inputs product details, total order cost is calculated dynamically, discount policies are applied based on conditions, and a detailed invoice is generated.
3	Write a Java program demonstrating method overloading to compute power and absolute value for different data types using static methods from the Math class.
4	Write a Java program to implement a Library Management System that allows adding, issuing, and returning books, tracks the total number of books using a static field, and provides book details and total book count using static methods.
5	Develop a Java program to perform array operations including displaying elements, finding maximum and minimum values, calculating sum and average, and searching for a specific element.
6	Develop a Java program to implement a simple hotel room booking system using two dimensional arrays that allows viewing room status, booking rooms by floor and room number, and exiting the system.

List of Assignment (Part C - Any 2)	
Sr. No	**Assignment**
1	Create a Java application that connects to a MySQL/ Postgre SQL database to manage employee information. Implement functionalities like adding, updating, deleting, and viewing employee records using JDBC. Use prepared statements to prevent SQL injection and handle exceptions gracefully.
2	Develop a Java application that connects to a MySQL/ Postgre SQL database to manage student information. Implement CRUD operations for student records using JDBC. Use prepared statements to handle SQL queries securely and ensure proper transaction management.
3	Create a Java application that reads weather data from a CSV file and performs basic statistical operations using Apache Commons Math. Use Java FX to create interactive charts and graphs to visualize temperature trends, humidity levels, and other weather parameters
4	Develop a Java application that reads patient data from a CSV file and calculates basic statistics such as average age, median heart rate, and standard deviation of blood pressure using the Apache Commons Math library. Implement functionality to visualize the data using Java FX charts
Sr. No	**Mini Project**
1	Banking system having the following operations: a. Create an account b. Deposit money c. Withdraw money d. Honor daily withdrawal limit e. Check the balance f. Display Account information.
2	Inventory management system having the following operations: a. List of all products b. Display individual product information c. Purchase d. Shipping e. Balance stock f. Loss and Profit calculation.

PART A
(Any 4)

Part A (1)

Experiment Title: Implement a robust Java calculator program that captures user input dynamically, processes mathematical expressions using conditional logic and looping constructs, and ensures efficient error handling.

Objective: Develop proficiency in Java control structures by implementing a dynamic calculator that processes user inputs for addition, subtraction, multiplication, division, and modulus while handling errors gracefully. Students learn to create robust, user-friendly console applications.

Prerequisites: Basic Java syntax including variables, data types (double, char, String), and input/output. Familiarity with Scanner class for reading inputs and understanding program flow control.

Tools Used: JDK 8 or higher (Eclipse IDE, IntelliJ, or command-line javac/java). Scanner from java.util package for dynamic input capture.

Outcomes: Gain competencies in conditional decision-making, loop-based repetition for multi-operation sessions, input validation, and exception management for real-world programming resilience.

Theory:

Control Statements and Program Flow

Control statements are essential parts of any programming language because they determine how instructions are executed. They allow a program to make decisions, repeat tasks, and manage its overall direction. Without these statements, every line would run sequentially, making programs rigid and non-interactive. The three major categories of control statements are selection, iteration, and jump statements.

Selection statements

Selection statements mainly if-else and switch help programs make choices based on certain conditions. The if-else statement compares values and executes specific blocks of code depending on whether a condition is true or false. For example, it can display “Pass” if marks are above 40 or “Fail” otherwise.

When several possible outcomes exist, the switch-case structure works better than multiple if-else checks. It compares a variable against several case labels (usually integers or characters) and executes the matching block. This makes code easier to read and manage, especially in menu-driven applications or multi-choice logic.

Iteration statements

Iteration allows certain sections of code to repeat until a condition changes ideal for tasks involving repetition like summing numbers, validating input, or displaying sequences. The while, do-while, and for loops are the main types:

while checks the condition before execution (pre-test).
do-while checks after execution, ensuring one run even if false initially.
for loop is compact and ideal when the number of iterations is known, such as counting from 1 to 10.
These loops reduce code duplication and make programs more efficient.

Jump statements
Loop control is further refined using break and continue.
break ends the loop immediately when a specific condition is met, preventing unnecessary iterations.
continue skips the remaining code for the current iteration but continues the loop with the next cycle.
Both statements make loop execution more flexible and controlled.

Scanner Class and Input Handling
Interactive programs require user input, and Java provides the Scanner class for this purpose. It reads input from the keyboard and converts it into usable data types using methods such as:
nextInt() for integers
nextDouble() for decimals
nextLine() for strings

However, user input can be unpredictable. Entering text where a number is expected may cause an error. To handle such cases, developers use exception handling.

Error Handling with Try-Catch-Finally
The try-catch-finally structure ensures programs run smoothly even when unexpected errors occur. Risky code goes inside the try block, and if an error appears, the catch block handles it gracefully instead of crashing the program. The finally block executes afterward, regardless of errors, often used for cleanup tasks.

Common exceptions include:

InputMismatchException – triggered when the Scanner input type doesn't match (e.g., entering text instead of a number).
ArithmeticException – raised when performing invalid arithmetic operations like dividing by zero.
Handling these ensures the program remains stable and user-friendly.

Integrating the Concepts
A well-structured Java program often combines all these features. It might use loops for repeated input, selection statements for decisions, try-catch for managing invalid entries, and jump statements for fine-tuned control. Together, they give programs flexibility, reliability, and a human-like ability to handle real-world interactions.

Code:

```
import java.util.*;

public class EngineeringCalculator {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== Engineering Calculator ===");

        while (true) {
            try {
                System.out.print("\nEnter expression (or type 'exit'
to quit): ");
                String input = sc.nextLine();

                if (input.equalsIgnoreCase("exit")) {
                    System.out.println("Calculator closed.");
                    break;
                }

                double result = evaluateExpression(input);
                System.out.println("Result = " + result);

            } catch (ArithmeticException ae) {
                System.out.println("Math Error: " + ae.getMessage());
            } catch (Exception e) {
                System.out.println("Invalid Expression. Please try
again.");
            }
        }
        sc.close();
    }

    // Method to evaluate full expression
    public static double evaluateExpression(String expression) {
        Stack<Double> values = new Stack<>();
        Stack<Character> operators = new Stack<>();

        for (int i = 0; i < expression.length(); i++) {

            char ch = expression.charAt(i);

            if (Character.isWhitespace(ch))
                continue;

            if (Character.isDigit(ch)) {
                StringBuilder sb = new StringBuilder();
```

```
                while (i < expression.length() &&
                       (Character.isDigit(expression.charAt(i)) ||
                        expression.charAt(i) == '.')) {
                    sb.append(expression.charAt(i));
                    i++;
                }

                values.push(Double.parseDouble(sb.toString()));
                i--;
            }

            else if (ch == '(') {
                operators.push(ch);
            }

            else if (ch == ')') {
                while (operators.peek() != '(') {
                    values.push(applyOperation(
                            operators.pop(),
                            values.pop(),
                            values.pop()));
                }
                operators.pop();
            }

            else if (isOperator(ch)) {

                while (!operators.empty() &&
                        precedence(ch) <=
precedence(operators.peek())) {
                    values.push(applyOperation(
                            operators.pop(),
                            values.pop(),
                            values.pop()));
                }

                operators.push(ch);
            }
        }

        while (!operators.empty()) {
            values.push(applyOperation(
                    operators.pop(),
                    values.pop(),
                    values.pop()));
        }

        return values.pop();
```

```
    }

    // Check valid operator
    public static boolean isOperator(char ch) {
        return ch == '+' || ch == '-' ||
               ch == '*' || ch == '/';
    }

    // Operator precedence
    public static int precedence(char op) {
        if (op == '+' || op == '-') return 1;
        if (op == '*' || op == '/') return 2;
        return 0;
    }

    // Apply operation
    public static double applyOperation(char op,
                                        double b,
                                        double a) {

        switch (op) {
            case '+': return a + b;
            case '-': return a - b;
            case '*': return a * b;
            case '/':
                if (b == 0)
                    throw new ArithmeticException("Division by zero");
                return a / b;
        }
        return 0;
    }
}
```

Output:

```
=== Engineering Calculator ===

Enter expression (or type 'exit' to quit): 10+5
Result = 15.0

Enter expression (or type 'exit' to quit): 20-3*2
Result = 14.0

Enter expression (or type 'exit' to quit): (4+6)*3
Result = 30.0

Enter expression (or type 'exit' to quit): 25/5
Result = 5.0
```

FAQs

Q: Why use switch over if-else?
A: Switch is cleaner for char ops, faster for exact matches, and avoids long chains.

Q: Program loops forever on bad input?
A: No – try-catch + sc.nextLine() clears invalid input; inner while reprompts safely.

Q: How to handle decimals precisely?
A: Use double; printf "%.2f" limits output digits. For money, consider BigDecimal later.

Q: Add more operations?
A: Extend switch cases (e.g., '^' for power via Math.pow). Keep validation consistent.

Q: Scanner not reading after nextDouble()?
A: Add sc.nextLine() to consume newline leftover.

Part A (2)

Experiment Title: Develop a Java program for an e commerce order processing system where products are initialized using multiple constructors, user inputs product details, total order cost is calculated dynamically, discount policies are applied based on conditions, and a detailed invoice is generated.

Objective: Master Java OOP concepts by creating an e-commerce order system that initializes products via multiple constructors, accepts user input, computes dynamic totals with conditional discounts, and generates professional invoices.

Prerequisites: Understanding of Java classes, constructor overloading, arrays, Scanner for input, conditional statements (if-else), loops (for), and basic methods with return types.

Tools Used: JDK 8+, any IDE (Eclipse/IntelliJ), or command-line (javac/java). Scanner class from java.util for runtime product input.

Outcomes: Acquire skills in constructor overloading for flexible object creation, dynamic array management for orders, conditional discount application, and formatted invoice output for business applications.

Theory:

Constructors and Dynamic Order Management

In object-oriented programming, a constructor is a special method used to initialize objects when they are created. It ensures that newly created objects start with meaningful or default values. Constructors carry the same name as the class and do not return any value.

A default constructor (no-arguments constructor) automatically assigns default or preset values to data members when no input is provided. For example, an item's name might default to "Unknown", its price to 0.0, and its quantity to 0. On the other hand, overloaded constructors accept one or more parameters (such as name, price, and qty) allowing objects to be initialized with customized data. This flexibility supports the creation of a variety of product objects with different properties while maintaining consistent initialization logic.

To manage multiple product details effectively, developers often use arrays or ArrayLists to store a collection of objects. Arrays provide a fixed-size structure suitable when the number of items is known, while ArrayLists are dynamic, allowing the program to add or remove elements during execution. This makes them ideal for building flexible systems such as online order processing or shop billing programs, where the number of orders can vary.

Essential business operations are typically handled through methods within the class. The calculateTotal() method, for example, multiplies price * quantity for each item and adds up all subtotals to produce the final amount. Another method, applyDiscount(), uses decision-making statements such as if-else to apply a discount only when total purchases exceed a given threshold for instance, "apply a 10% discount if total > ₹5000, else no discount." This approach integrates conditional logic directly into the billing process, ensuring fairness and consistency.

To produce professional output, many programs use formatted printing techniques such as System.out.printf() or the String.format() method. These statements align text, control decimal precision, and display values neatly, creating a clear and readable invoice format for end-users. For example, the invoice may show columns for Item Name, Quantity, Unit Price, and Total Amount much like a real-world bill.

Lastly, dynamic input through loops enables scalable and interactive systems. Loops allow users to repeatedly enter product details until they choose to stop, automatically adding each item to an ArrayList. This structure not only supports flexible workflows but also demonstrates key programming principles iteration, conditional logic, and object initialization working together to form a complete, real-world application.

Code:

```
import java.util.ArrayList;
import java.util.Scanner;

class Product {
    private String name;
    private double price;
    private int quantity;
    private String category;

    // Constructor 1
    public Product(String name, double price) {
        this.name = name;
        this.price = price;
        this.quantity = 1;
        this.category = "General";
    }

    // Constructor 2
    public Product(String name, double price, int quantity) {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
        this.category = "General";
    }

    // Constructor 3
    public Product(String name, double price, int quantity, String
category) {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
        this.category = category;
    }
```

```
    public double getTotalPrice() {
        return price * quantity;
    }

    public String getCategory() {
        return category;
    }

    public String toString() {
        return name + "\t" + price + "\t" + quantity + "\t" +
getTotalPrice();
    }
}

class Order {
    private ArrayList<Product> productList = new ArrayList<>();

    public void addProduct(Product p) {
        productList.add(p);
    }

    public double calculateTotal() {
        double total = 0;
        for (Product p : productList) {
            total += p.getTotalPrice();
        }
        return total;
    }

    public double applyDiscount(double total) {
        double discount = 0;

        if (total > 5000) {
            discount = total * 0.15;   // 15% discount
        } else if (total > 3000) {
            discount = total * 0.10;   // 10% discount
        } else if (total > 1000) {
            discount = total * 0.05;   // 5% discount
        }

        return discount;
    }

    public void generateInvoice() {
        double total = calculateTotal();
        double discount = applyDiscount(total);
        double finalAmount = total - discount;
```

```
        System.out.println("\n===== INVOICE =====");
        System.out.println("Product\tPrice\tQty\tTotal");
        for (Product p : productList) {
            System.out.println(p);
        }

        System.out.println("------------------------------");
        System.out.println("Total Amount: " + total);
        System.out.println("Discount: " + discount);
        System.out.println("Final Payable Amount: " + finalAmount);
        System.out.println("==============================");
    }
}

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Order order = new Order();

        // Predefined products using different constructors
        Product p1 = new Product("Laptop", 45000);
        Product p2 = new Product("Mouse", 500, 2);
        Product p3 = new Product("Headphones", 1500, 1,
"Electronics");

        order.addProduct(p1);
        order.addProduct(p2);
        order.addProduct(p3);

        System.out.print("Enter number of additional products: ");
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter product details:");
            System.out.print("Name: ");
            String name = sc.nextLine();

            System.out.print("Price: ");
            double price = sc.nextDouble();

            System.out.print("Quantity: ");
            int qty = sc.nextInt();
            sc.nextLine();

            System.out.print("Category: ");
            String category = sc.nextLine();
```

```
            Product userProduct = new Product(name, price, qty,
category);
            order.addProduct(userProduct);
        }

        order.generateInvoice();
        sc.close();
    }
}
```

Output:

```
Enter number of additional products: 1

Enter product details:
Name: Keyboard
Price: 1200
Quantity: 1
Category: Electronics

===== INVOICE =====
Product Price Qty Total
Laptop 45000.0 1 45000.0
Mouse 500.0 2 1000.0
Headphones 1500.0 1 1500.0
Keyboard 1200.0 1 1200.0
----------------------------
Total Amount: 48700.0
Discount: 7305.0
Final Payable Amount: 41395.0
============================
```

FAQs

Q: Why multiple constructors?
A: Flexibility – pre-load inventory (3-param), quick add (2-param), safe defaults.

Q: Array vs ArrayList?
A: Fixed-size array for lab simplicity; ArrayList for dynamic growth in production.

Q: Discount not applying?
A: Check total thresholds exactly; use >2000.00, not >=. Test with ₹2001.

Q: User input skips quantity?
A: nextDouble() leaves newline; nextInt() handles it fine here.

Q: Extend for more features?
A: Add removeProduct(), tax calculation, or ArrayList<Product> for unlimited items.

Part A (3)

Experiment Title: Write a Java program demonstrating method overloading to compute power and absolute value for different data types using static methods from the Math class.

Objective: Understand method overloading by creating power and absolute value methods for multiple data types, then compare results with Math class static methods to appreciate Java's built-in precision handling.

Prerequisites: Basic Java syntax, primitive data types (int, double, float), method declaration, return statements, and familiarity with Math class static methods.

Tools Used: JDK 8+, any Java IDE or command-line compiler. No external libraries needed beyond core java.lang.Math.

Outcomes: Master method overloading resolution rules, implement mathematical computations manually vs library calls, and gain insight into data type promotion and precision differences.

Theory:

Method Overloading and the Math Class

Method overloading is an important feature of object-oriented programming that allows multiple methods in the same class to share the same name but differ in the number or type of parameters. This enables code readability and flexibility while allowing the programmer to define various behaviors for similar operations. Overloading is resolved at compile-time, meaning the compiler determines which version of the method to call based on the arguments supplied by the user.

For instance, a method named display() might have several versions: one that prints a string, another that prints an integer, and one that prints both. The compiler automatically chooses the matching version depending on the argument type. However, it’s important to note that return types do not influence overloading. Two methods cannot be considered overloaded if their only difference lies in their return type there must be a distinct parameter list.

A practical example of overloading can be seen in mathematical operations. Java’s built-in Math class contains many static methods that perform general-purpose mathematical calculations. Since they are static, they can be accessed directly via the class name, without creating an object for example:

Math.pow(base, exp) calculates a number raised to a power.

Math.abs(n) returns the absolute (non-negative) value of a number, regardless of whether it’s positive or negative.

The Math class provides versions of these methods for different numeric types (like int, float, and double) through method overloading. For example, both Math.abs(int n) and Math.abs(double n) exist, ensuring correct data handling according to the type of the argument.

In user-defined classes, programmers can create their own overloaded methods to handle different data types. Suppose we define two custom methods:

int power(int base, int exp) for integer exponentiation.

double power(double base, double exp) for real-number exponentiation.

When a programmer calls power(2, 3), the integer version runs, whereas power(2.0, 3.0) invokes the double version. This ensures type safety the compiler can match the correct method version to the input type, avoiding unnecessary type conversions or ambiguity.

Code:

```
import java.util.Scanner;

class Calculator {

    // Method Overloading for Power
    public static double power(int base, int exponent) {
        return Math.pow(base, exponent);
    }

    public static double power(double base, double exponent) {
        return Math.pow(base, exponent);
    }

    public static double power(float base, int exponent) {
        return Math.pow(base, exponent);
    }

    // Method Overloading for Absolute Value
    public static int absolute(int num) {
        return Math.abs(num);
    }

    public static double absolute(double num) {
        return Math.abs(num);
    }

    public static long absolute(long num) {
        return Math.abs(num);
    }
}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("=== POWER CALCULATION ===");
        System.out.print("Enter integer base: ");
        int ibase = sc.nextInt();
```

```
        System.out.print("Enter integer exponent: ");
        int iexp = sc.nextInt();
        System.out.println("Result: " + Calculator.power(ibase,
iexp));

        System.out.print("\nEnter double base: ");
        double dbase = sc.nextDouble();
        System.out.print("Enter double exponent: ");
        double dexp = sc.nextDouble();
        System.out.println("Result: " + Calculator.power(dbase,
dexp));

        System.out.println("\n=== ABSOLUTE VALUE ===");
        System.out.print("Enter integer: ");
        int inum = sc.nextInt();
        System.out.println("Absolute: " + Calculator.absolute(inum));

        System.out.print("Enter double: ");
        double dnum = sc.nextDouble();
        System.out.println("Absolute: " + Calculator.absolute(dnum));

        System.out.print("Enter long: ");
        long lnum = sc.nextLong();
        System.out.println("Absolute: " + Calculator.absolute(lnum));

        sc.close();
    }
}
```

Output:

```
=== POWER CALCULATION ===
Enter integer base: 2
Enter integer exponent: 3
Result: 8.0

Enter double base: 2.5
Enter double exponent: 2
Result: 6.25

=== ABSOLUTE VALUE ===
Enter integer: -10
Absolute: 10
Enter double: -15.7
Absolute: 15.7
Enter long: -200
Absolute: 200
```

FAQs

Q: Why use Math.pow() in double overload?
A: Handles fractional exponents (2^0.5) and edge cases with precision; custom loops limited to integers.

Q: Can overload by return type only?
A: No Java requires parameter list differences. Return type ignored in resolution.

Q: What if ambiguous call like power(2, 3.0)?
A: Compile error. Compiler can't choose int vs double. Use exact types.

Q: Integer power(2, 31) overflows?
A: Yes, int max ~2^31-1. Use long or double for large exponents.

Q: Add long overload?
A: Yes: static long power(long base, long exp) with loop, compare Math.pow() cast.

Part A (5)

Title: Develop a Java program to perform array operations including displaying elements, finding maximum and minimum values, calculating sum and average, and searching for a specific element.
Objective: Perform array operations: display, max/min, sum/average, linear search.

Learning Outcomes: Implement enhanced for-loops/indices for traversal, O(n) algorithms for max/min/sum/search, floating-point average calc, user-defined array size/input. Essential for data analysis in AI/ERP.

Theory:

Arrays and Basic Operations
An array is a fundamental data structure that stores multiple values of the same data type in a single container. Each element in an array occupies a specific position, identified by an index, starting from 0. This means the first element is accessed as array[0], the second as array[1], and so on. Arrays have a fixed size, meaning the total number of elements must be defined when the array is created. For instance, int[] marks = new int[5]; creates an array of five integer elements indexed from 0 to 4.

Because arrays store related data in an ordered sequence, they are ideal for performing repetitive operations like searching, summing, or finding maximum and minimum values.

Linear Search
The simplest way to find a particular value in an array is through a linear search. In this method, each element is checked one by one until a match is found or the end of the array is reached. Although it is not the fastest technique for large data sets, linear search works efficiently for small or unsorted arrays. For example, in a list of student roll numbers, the program compares each roll number with the target value until the desired one matches.

Finding Maximum and Minimum
To determine the maximum or minimum value in an array, an extreme value is first assumed. For maximum search, the initial extreme can be the first element of the array. The algorithm then loops through the remaining elements, comparing each with the current extreme. If a higher value appears, it replaces the stored maximum. Similarly, for finding the minimum, the smallest value is tracked in each iteration. This simple comparison-based approach ensures that by the end of the loop, the true highest or lowest value is found.

Calculating Sum and Average
Arrays also make it easy to compute the sum of elements using an accumulator variable. The accumulator starts at zero and adds each element's value during loop iterations. Once the total sum is obtained, the average can be calculated by dividing this sum by the total number of elements in the array. For instance, if the array stores marks of five students, the program sums all marks and divides the total by 5 to get the class average.

Code:

```
import java.util.Scanner;
public class Main {
    // Display Array
    public static void display(int[] arr) {
        System.out.println("Array Elements:");
        for (int num : arr) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
    // Find Maximum
    public static int findMax(int[] arr) {
        int max = arr[0];
        for (int num : arr) {
            if (num > max) {
                max = num;
            }
        }
        return max;
    }
    // Find Minimum
    public static int findMin(int[] arr) {
        int min = arr[0];
        for (int num : arr) {
            if (num < min) {
                min = num;
            }
        }
        return min;
    }
    // Calculate Sum
    public static int calculateSum(int[] arr) {
        int sum = 0;
        for (int num : arr) {
            sum += num;
        }
        return sum;
    }
    // Search Element
    public static int searchElement(int[] arr, int key) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) {
                return i; // return index
            }
        }
        return -1; // not found
    }
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of elements: ");
    int n = sc.nextInt();

    int[] arr = new int[n];

    System.out.println("Enter elements:");
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }

    int choice;

    do {
        System.out.println("\n===== ARRAY MENU =====");
        System.out.println("1. Display Elements");
        System.out.println("2. Find Maximum");
        System.out.println("3. Find Minimum");
        System.out.println("4. Calculate Sum and Average");
        System.out.println("5. Search Element");
        System.out.println("6. Exit");
        System.out.print("Enter choice: ");
        choice = sc.nextInt();

        switch (choice) {

            case 1:
                display(arr);
                break;

            case 2:
                System.out.println("Maximum: " + findMax(arr));
                break;

            case 3:
                System.out.println("Minimum: " + findMin(arr));
                break;

            case 4:
                int sum = calculateSum(arr);
                double avg = (double) sum / arr.length;
                System.out.println("Sum: " + sum);
                System.out.println("Average: " + avg);
                break;

            case 5:
                System.out.print("Enter element to search: ");
```

```
                    int key = sc.nextInt();
                    int index = searchElement(arr, key);
                    if (index != -1) {
                        System.out.println("Element found at index: "
+ index);
                    } else {
                        System.out.println("Element not found.");
                    }
                    break;

                case 6:
                    System.out.println("Exiting...");
                    break;

                default:
                    System.out.println("Invalid choice.");
            }

        } while (choice != 6);

        sc.close();
    }
}
```

Output:

```
Enter number of elements: 6
Enter elements:
12 7 25 4 18 10

===== ARRAY MENU =====
1. Display Elements
2. Find Maximum
3. Find Minimum
4. Calculate Sum and Average
5. Search Element
6. Exit
Enter choice: 1
Array Elements:
12 7 25 4 18 10

===== ARRAY MENU =====
1. Display Elements
2. Find Maximum
3. Find Minimum
4. Calculate Sum and Average
5. Search Element
6. Exit
Enter choice: 2
Maximum: 25
```

```
===== ARRAY MENU =====
1. Display Elements
2. Find Maximum
3. Find Minimum
4. Calculate Sum and Average
5. Search Element
6. Exit
Enter choice: 3
Minimum: 4

===== ARRAY MENU =====
1. Display Elements
2. Find Maximum
3. Find Minimum
4. Calculate Sum and Average
5. Search Element
6. Exit
Enter choice: 4
Sum: 76
Average: 12.666666666666666

===== ARRAY MENU =====
1. Display Elements
2. Find Maximum
3. Find Minimum
4. Calculate Sum and Average
5. Search Element
6. Exit
Enter choice: 5
Enter element to search: 4
Element found at index: 3

===== ARRAY MENU =====
1. Display Elements
2. Find Maximum
3. Find Minimum
4. Calculate Sum and Average
5. Search Element
6. Exit
Enter choice: 6
Exiting...

=== Code Execution Successful ===
```

PART B
(Any 4)

Part B (1)

Experiment Title: Create a Java program demonstrating single inheritance where a subclass extends a superclass and calls its methods.

Objective: Understand single inheritance by creating a subclass that extends a superclass, inherits its methods and fields, and demonstrates method calling from the parent class. Master the extends keyword and constructor chaining for real-world OOP applications.

Prerequisites

Basic knowledge of Java classes, objects, constructors, and methods. Familiarity with creating class files and running Java programs using command line or IDE. Understanding of access modifiers (public, private) is helpful but not mandatory.

Tools Used

1. Java Development Kit (JDK 8 or higher)
2. Text editor (Notepad++, VS Code) or IDE (Eclipse, IntelliJ IDEA, NetBeans)
3. Command line compiler (javac, java) or IDE run button

Outcomes

1. Implement single inheritance hierarchy with proper method inheritance and calling.
2. Use super keyword for constructor chaining and method access.
3. Demonstrate code reuse and "is-a" relationships in OOP design.
4. Debug common inheritance errors like missing extends or access issues.

Theory:

Imagine you're building a zoo management system. Every animal in the zoo shares common traits they all have a name, age, and basic behaviors like eating or sleeping. But each animal type adds unique behaviors. A lion roars, a bird flies, a dog barks. Creating separate classes for each animal from scratch would repeat the common code endlessly. That's where inheritance saves the day!

Single Inheritance is like family inheritance your child class (subclass/derived class) gets all the traits and behaviors from one parent class (superclass/base class). Java uses the extends keyword to create this "is-a" relationship. A Dog is-a Animal, so Dog inherits Animal's fields and methods automatically.

Syntax:

```
class SuperClass {  // Parent class
    // fields, constructors, methods
}
class SubClass extends SuperClass {  // Child class inherits
everything
    // can add new fields/methods
    // can override parent's methods
}
```

Key Concepts Explanation:

1.Fields Inheritance: Subclass gets parent's non-private fields for free. Dog dog = new Dog(); dog.name = "Buddy"; works even if name is in Animal class.

2.Method Inheritance: Subclass can call parent's methods directly: dog.eat(); calls Animal's eat() method.

3.Constructor Chaining: When you create a subclass object, Java automatically calls the superclass constructor first using super(). If not explicit, it calls the no-arg superclass constructor.

- super Keyword: Three superpowers!
- super(): Call parent constructor (must be first statement)
- super.method(): Call parent's version of overridden method
- super.field: Access parent's field if shadowed

Algorithm:

1. Define superclass Animal with fields (name, age), constructor, and eat() method.
2. Define subclass Dog extending Animal with additional breed field and bark() method.
3. In Dog constructor, call super(name, age) for parent initialization.
4. In main(), create Dog object and call inherited eat() + new bark() methods.
5. Demonstrate overriding by calling eat() (shows enhanced version).

Code:

```
class Animal {

    protected String name;
    protected int age;

    // Constructor of Superclass
    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void eat() {
        System.out.println(name + " is eating.");
    }

    public void sleep() {
        System.out.println(name + " is sleeping.");
    }
}
```

```
// Subclass
class Dog extends Animal {

    private String breed;

    // Constructor of Subclass
    public Dog(String name, int age, String breed) {
        super(name, age);   // Call superclass constructor
        this.breed = breed;
    }

    public void bark() {
        System.out.println(name + " is barking.");
    }

    // Method overriding
    @Override
    public void eat() {
        System.out.println(name + " is eating dog food.");
    }

    public void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Breed: " + breed);
    }
}

public class Main {

    public static void main(String[] args) {

        Dog d1 = new Dog("Tommy", 3, "Labrador");

        d1.displayDetails();

        System.out.println();

        d1.eat();     // Overridden method
        d1.sleep();   // Inherited method
        d1.bark();    // Subclass method
    }
}
```

Output:

```
Name: Tommy
Age: 3
Breed: Labrador

Tommy is eating dog food.
Tommy is sleeping.
Tommy is barking.

=== Code Execution Successful ===
```

FAQs

Q1: "Compile error: cannot find symbol 'Animal'"?
A: Compile superclass first: javac Animal.java then subclass. Or use IDE project structure.

Q2: "super() must be first statement" error?
A: Yes! Constructor chaining rule. Move super() to line 1 in subclass constructor.

Q3: Subclass can't access superclass private field?
A: Use protected instead of private for inheritance access, or public getters/setters.

Q4: How to call original eat() after overriding?
A: super.eat() inside overridden method as shown in Dog.eat().

Q5: Multiple files or single file OK?
A: Multiple preferred for organization. Single file works if main class is public and matches filename.

Part B (2)

Experiment Title: Implement an interface in Java and create multiple classes that implement the interface to demonstrate polymorphism.

Objective: Master Java interfaces by implementing a common contract across multiple classes and demonstrate polymorphism where interface references invoke different class implementations dynamically. Understand how interfaces enable flexible, extensible OOP design.

Prerequisites: Basic knowledge of Java classes, methods, and inheritance. Familiarity with abstract methods and object references. Understanding that interfaces define "what" without "how" (pure contracts).

Tools Used:

1. Java Development Kit (JDK 8 or higher)
2. Text editor (Notepad++, VS Code) or IDE (Eclipse, IntelliJ IDEA, NetBeans)
3. Command line compiler (javac, java) or IDE run button

Outcomes:

1. Implement interface contracts using implements keyword across multiple classes.
2. Demonstrate polymorphism with interface array references calling different implementations.
3. Use interfaces for loose coupling and extensibility in OOP design.
4. Debug common interface errors like missing method implementations.

Theory:

Think of interfaces like job contracts. A company posts a "Driver" job requiring drive() and brake() methods. Different people (TruckDriver, CarDriver, BikeDriver) apply and implement these required skills differently. The HR system doesn't care how they drive it just calls drive() on any Driver reference, and the right implementation runs automatically. That's polymorphism in action!

Java Interfaces define pure contracts methods without bodies that implementing classes must provide. Unlike classes, interfaces can't be instantiated and contain no concrete implementation (pre-Java 8). They're the ultimate "is-able-to" relationship.

Syntax:

```
interface InterfaceName {
    // All methods public abstract by default (no body)
    void method1();
    int method2(int param);

    // Constants: public static final double PI = 3.14;
}
class MyClass implements InterfaceName {
    // MUST implement ALL interface methods
    public void method1() { /* implementation */ }
    public int method2(int param) { /* implementation */ }
}
```

Polymorphism Magic:

```
Drawable[] shapes = {new Circle(), new Rectangle()};
for(Drawable shape : shapes) {
    shape.draw();     //  Calls  Circle.draw()  OR  Rectangle.draw()
dynamically!
}
```

Key Interface:

- **Multiple Implementation:** One interface, many classes: class Circle implements Drawable, class Square implements Drawable.
- **Interface References:** Drawable d = new Circle(); d.draw(); calls Circle's version!
- **Array Polymorphism:** Store different implementations in one array, loop calls correct method each time.
- **Loose Coupling:** Code works with any Drawable without knowing concrete class.

Algorithm:

1. Define Drawable interface with draw() abstract method.
2. Create Circle class implementing Drawable with radius-based draw().
3. Create Rectangle class implementing Drawable with width/height draw().
4. In main(), create interface array with Circle and Rectangle objects.
5. Loop through array calling draw()—demonstrates polymorphic behavior.

Code :

```
// Interface
interface Shape {
    void draw();
    double calculateArea();
}

// Class 1
class Circle implements Shape {

    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }
```

```
    public void draw() {
        System.out.println("Drawing Circle");
    }

    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}

// Class 2
class Rectangle implements Shape {

    private double length;
    private double width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    public void draw() {
        System.out.println("Drawing Rectangle");
    }

    public double calculateArea() {
        return length * width;
    }
}

public class Main {

    public static void main(String[] args) {

        // Polymorphism using interface reference
        Shape s1 = new Circle(5);
        Shape s2 = new Rectangle(4, 6);

        s1.draw();
        System.out.println("Area: " + s1.calculateArea());

        System.out.println();

        s2.draw();
        System.out.println("Area: " + s2.calculateArea());
    }
}
```

Output:

```
Drawing Circle
Area: 78.53981633974483

Drawing Rectangle
Area: 24.0

=== Code Execution Successful ===
```

FAQs:

Q1: "Class must implement inherited abstract method" error?
A: Missing draw() method or wrong signature. Check spelling, parameters, return type match exactly.

Q2: Can interface have fields?
A: Only public static final constants. No instance variables—pure contract, no state.

Q3: Drawable d = new Drawable() fails?
A: Interfaces can't be instantiated. Always new ConcreteClass().

Q4: Multiple interfaces possible?
A: Yes! class MyClass implements Drawable, Resizable, Colorable.

Q5: Abstract class vs Interface for polymorphism?
A: Interface for multiple inheritance + pure contracts. Abstract for shared code + "is-a"

Part B (3)

Experiment Title: Write a Java program to create an abstract class with an abstract method and extend it in a subclass that provides its implementation.

Objective: Master abstract classes by creating a partial blueprint with concrete methods and abstract methods that subclasses must implement. Demonstrate inheritance hierarchy where subclasses provide missing functionality while inheriting shared code.

Prerequisites: Basic knowledge of Java classes, inheritance, and interfaces. Understanding of method overriding and constructor chaining. Familiarity with access modifiers and abstract concepts.

Tools Used:

1. Java Development Kit (JDK 8 or higher)
2. Text editor (Notepad++, VS Code) or IDE (Eclipse, IntelliJ IDEA, NetBeans)
3. Command line compiler (javac, java) or IDE run button

Outcomes:

1. Create abstract classes with both concrete and abstract methods.
2. Implement abstract methods in concrete subclasses.
3. Demonstrate code reuse through shared abstract class functionality.
4. Understand abstract class instantiation rules and subclass requirements.

Theory:

Imagine you're designing a graphics engine for a game. All shapes share common behaviors they can move, resize, and display info but each shape calculates area differently (circle uses πr^2, rectangle uses width × height). Creating a plain Shape class won't work because area calculation varies. An abstract class is the perfect middle ground!

Abstract Classes are partial blueprints half complete, half "fill in the blanks." They contain:

- Concrete methods (shared functionality)
- Abstract methods (must be implemented by subclasses)
- Fields and constructors

Syntax:

```
abstract class AbstractClassName {
    // Concrete fields
    protected double x, y;

    // Concrete constructor
    public AbstractClassName(double x, double y) {
        this.x = x; this.y = y;
    }

    // Concrete method - shared by all subclasses
```

```
    public void move(int dx, int dy) {
        x += dx; y += dy;
    }

    // Abstract method - NO BODY! Subclasses must implement
    abstract double calculateArea();  // Semicolon only
}
```

Abstract Class Rules:

- Can't Instantiate: new Shape() → Compile error!
- Must Extend: Concrete subclasses use extends Shape
- Abstract Methods: Subclasses MUST implement ALL abstract methods
- Partial Implementation: Perfect for "is-a" with customization

Abstract vs Concrete vs Interface Comparison:

Feature	Abstract Class	Concrete Class	Interface
Instantiation	No	Yes	No
Constructors	Yes	Yes	No
Fields	Yes (any)	Yes	Constants only
Methods	Concrete + Abstract	Concrete only	Abstract (default Java 8+)
Inheritance	Single	Single	Multiple
Purpose	Partial "is-a"	Complete "is-a"	"can-do" ability

Algorithm:

1. Define abstract Shape class with concrete displayInfo(), abstract calculateArea().
2. Create Circle subclass extending Shape implementing calculateArea() for πr^2.
3. Create Rectangle subclass extending Shape implementing calculateArea() for w×h.
4. In main(), create Shape array with Circle/Rectangle objects.
5. Call polymorphic methods demonstrating abstract class power.

Code:

```
// Abstract class
abstract class Employee {

    protected String name;
    protected double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
```

```
    // Abstract method
    public abstract double calculateBonus();

    // Normal method
    public void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
    }
}

// Subclass
class Developer extends Employee {

    public Developer(String name, double salary) {
        super(name, salary);
    }

    // Implementation of abstract method
    @Override
    public double calculateBonus() {
        return salary * 0.10; // 10 percent bonus
    }
}

public class Main {

    public static void main(String[] args) {

        // Abstract class reference
        Employee emp = new Developer("Rahul", 50000);

        emp.displayDetails();

        double bonus = emp.calculateBonus();
        System.out.println("Bonus: " + bonus);
    }
}
```

Output:

```
Name: Rahul
Salary: 50000.0
Bonus: 5000.0

=== Code Execution Successful ===
```

FAQs:

Q1: "'Shape' is abstract; cannot be instantiated" error?
A: Correct behavior! Use new Circle() or new Rectangle(). Abstract classes can't be directly created.

Q2: "Must override abstract method calculateArea()" error?
A: Subclass missing @Override public double calculateArea() { ... }. Must match signature exactly.

Q3: Can abstract class have concrete methods?
A: Yes! That's the power shared code (displayInfo()) + customization (calculateArea()).

Q4: Abstract class vs Interface when to choose?
A: Abstract: shared code/state + "is-a". Interface: pure contract + multiple inheritance.

Q5: Can abstract class be final?
A: No final abstract contradicts (can't extend but must extend to be concrete).

Part B (4)

Experiment Title: Develop a Java application that simulates an ATM machine. Implement account balance check, withdrawal, and deposit. Use try, catch, and finally blocks to handle exceptions such as insufficient funds using Arithmetic Exception and invalid input using Illegal Argument Exception. Ensure smooth execution after exception handling.

Objective:
Develop a complete ATM simulation demonstrating exception handling for real-world errors: insufficient funds (ArithmeticException), invalid amounts (IllegalArgumentException). Ensure the application continues running smoothly after exceptions using proper try-catch-finally blocks.

Prerequisites:
Basic Java knowledge: classes, methods, Scanner input, if-else, loops. Understanding of method scope and return types. Familiarity with OOP concepts from previous labs.

Tools Used:
1. Java Development Kit (JDK 8 or higher)
2. Text editor (Notepad++, VS Code) or IDE (Eclipse, IntelliJ IDEA, NetBeans)
3. Command line compiler (javac, java) or IDE run button

Outcomes:
1. Implement custom exception handling for banking operations.
2. Use try-catch-finally for resource cleanup and error recovery.
3. Create user-friendly error messages with program continuity.
4. Understand exception propagation and multi-level handling.

Theory:
Picture yourself at an ATM. You enter PIN (valid), check balance ($500), withdraw $600 → ERROR: "Insufficient funds!" But the ATM doesn't crash it says "Transaction failed, try again?" and returns to main menu. That's professional exception handling!

Exceptions are Java's way of saying "Something went wrong, but I can recover gracefully." Instead of program crash, exceptions provide:

- Error type identification: ArithmeticException, IllegalArgumentException
- Recovery path: Return to menu, ask valid input
- Cleanup: finally block closes resources

Exception Handling Hierarchy:

Throwable

├── Error (OutOfMemoryError - don't handle)

└── Exception

├── RuntimeException (unchecked)

│ ├── ArithmeticException (÷0, negative withdrawal)

│ └── IllegalArgumentException (invalid input)

└── Checked Exception (IOException - must handle)

Try-Catch-Finally Syntax:

```
try {
    // Risky code: withdrawal, division, file I/O
    if(balance < amount) throw new ArithmeticException("Insufficient
funds");
} catch(ArithmeticException e) {
    // Handle math errors
    System.out.println("Error: " + e.getMessage());
} catch(IllegalArgumentException e) {
    // Handle bad input
    System.out.println("Invalid input: " + e.getMessage());
} finally {
    // ALWAYS executes: cleanup Scanner, log transaction
    System.out.println("Transaction processed. Choose next action.");
}
```

ATM-Specific Exception Strategy:

1. **Insufficient Funds:** Custom ArithmeticException when withdrawal > balance
2. **Invalid Amount:** IllegalArgumentException for negative/zero amounts
3. **Invalid Choice:** Menu input validation with loop recovery
4. **Continuous Execution:** Outer do-while loop survives inner exceptions

Algorithm:

1. Initialize ATM with 10000 balance, continuous menu loop.
2. Display menu: 1-Check Balance, 2-Deposit, 3-Withdraw, 4-Exit.
3. For each choice, wrap risky operations in try-catch-finally.
4. Withdraw: Validate amount > 0, then balance >= amount, else throw exceptions.
5. Catch specific exceptions with user messages, finally shows menu readiness.
6. Loop continues until user chooses Exit.

Code:

```
import java.util.Scanner;

class ATM {

    private double balance;

    public ATM(double initialBalance) {
        balance = initialBalance;
    }

    public void checkBalance() {
        System.out.println("Current Balance: " + balance);
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be
positive.");
        }
        balance += amount;
        System.out.println("Deposit successful.");
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must
be positive.");
        }
        if (amount > balance) {
            throw new ArithmeticException("Insufficient funds.");
        }
        balance -= amount;
        System.out.println("Withdrawal successful.");
    }
}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        ATM atm = new ATM(10000); // Initial balance
        int choice;

        do {
            System.out.println("\n===== ATM MENU =====");
            System.out.println("1. Check Balance");
```

```
            System.out.println("2. Deposit");
            System.out.println("3. Withdraw");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");

            choice = sc.nextInt();

            try {

                switch (choice) {

                    case 1:
                        atm.checkBalance();
                        break;

                    case 2:
                        System.out.print("Enter deposit amount: ");
                        double depositAmount = sc.nextDouble();
                        atm.deposit(depositAmount);
                        break;

                    case 3:
                        System.out.print("Enter withdrawal amount: ");
                        double withdrawAmount = sc.nextDouble();
                        atm.withdraw(withdrawAmount);
                        break;

                    case 4:
                        System.out.println("Thank you for using ATM.");
                        break;

                    default:
                        System.out.println("Invalid choice.");
                }

            } catch (IllegalArgumentException e) {
                System.out.println("Error: " + e.getMessage());

            } catch (ArithmeticException e) {
                System.out.println("Error: " + e.getMessage());

            } catch (Exception e) {
                System.out.println("Invalid input. Please enter correct
data.");
                sc.nextLine(); // clear buffer

            } finally {
                System.out.println("Transaction processed.");
            }
```

```
        } while (choice != 4);

        sc.close();
    }
}
```

Output:

```
===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter choice: 1
Current Balance: 10000.0
Transaction processed.

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter choice: 2
Enter deposit amount: 5000
Deposit successful.
Transaction processed.

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter choice: 3
Enter withdrawal amount: 2
Withdrawal successful.
Transaction processed.

===== ATM MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Enter choice: 1
Current Balance: 14998.0
Transaction processed.

===== ATM MENU =====
```

```
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
```

FAQs

Q1: "Exception in thread main" still appears?
A: Normal for uncaught exceptions. Our menu loop catches them program continues!

Q2: Scanner not reading after nextDouble()?
A: Fixed with input validation loop. scanner.next() clears bad input.

Q3: Why specific catch blocks order matters?
A: Specific first (Illegal Argument Exception), general last (Exception). Parent classes catch too early otherwise.

Q4: Can I add PIN validation?
A: Yes! Wrap in try-catch, throw IllegalArgumentException for wrong PIN x3.

Q5: Program too fast between transactions?
A: Add Thread.sleep (1500) in finally for realistic ATM pause effect.

PART C
(Any 2)

Part C (1)

Experiment Title: Create a Java application that connects to a MySQL/PostgreSQL database to manage employee information. Implement functionalities like adding, updating, deleting, and viewing employee records using JDBC. Use prepared statements to prevent SQL injection and handle exceptions gracefully.

Objective: Develop a complete Java console application that connects to MySQL database to manage employee records through core CRUD operations. Students will implement secure database interactions using JDBC prepared statements while handling real-world exceptions gracefully.

Learning Outcomes

Establish JDBC connection to MySQL and execute parameterized SQL queries safely.
Implement Add, Update, Delete, and View operations on employee data with proper error handling.
Understand SQL injection prevention and resource management in database applications.

Theory:

JDBC stands for Java Database Connectivity. Java Database Connectivity (JDBC) serves as the bridge between Java applications and relational databases like MySQL. Think of it as a universal translator your Java code speaks one language, the database another, and JDBC handles the conversation seamlessly. The process begins with loading the JDBC driver, a small piece of software that understands both sides.

The main steps involved are

1. Load the JDBC driver.
2. Establish a connection to the database.
3. Create a statement.
4. Execute SQL queries.
5. Process results.
6. Close the connection.

Establishing Database Connection

```
Connection is created using
DriverManager.getConnection(URL, username, password)
Example URL format
jdbc:mysql://localhost:3306/employeedb
This returns a Connection object that allows interaction with the
database.
```

PreparedStatement and Security

```
PreparedStatement is a pre compiled SQL statement. It uses
placeholders represented by question marks.
Example
INSERT INTO employees VALUES (?, ?, ?, ?)
Values are set using methods like
setInt()
setString()
setDouble()
Prepared statements prevent SQL injection because user input is
treated as data, not as SQL code.
Unsafe example
"SELECT * FROM employees WHERE id = " + userInput
If user enters malicious input, the database may be damaged.
Prepared statements remove this risk.
```

CRUD Operations

1. CREATE Operation

Used to insert new employee records.
Method used
executeUpdate()
It returns the number of rows affected.

Example action
Add employee name, department, and salary.

2. READ Operation

Used to retrieve employee records.
Method used
executeQuery()
It returns a ResultSet object.
Use next() to iterate through records.
Retrieve values using
getInt()
getString()
getDouble()

3. UPDATE Operation

Used to modify existing employee records.
Method used
executeUpdate()
Include WHERE clause to update specific record.

4. DELETE Operation

Used to remove employee records.
Method used
executeUpdate()

Example
DELETE FROM employees WHERE id = ?

Exception Handling

All database operations must be enclosed inside try catch blocks.
Catch SQLException to handle database related errors.

Useful methods
e.getMessage()
e.getSQLState()
e.getErrorCode()

Use try with resources to automatically close
Connection
PreparedStatement
ResultSet

This prevents memory leaks and connection issues.

Code:

1.Create a Database Table

```
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(100),
    salary DOUBLE
);
```

2. Java Program

```
import java.sql.*;
import java.util.Scanner;

public class Main {

    // Change according to your DB
    static final String URL ="jdbc:mysql://localhost:3306/testdb";
    static final String USER = "root";
    static final String PASSWORD = "password";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    public static void addEmployee() {
        String sql = "INSERT INTO employees (id, name, department,
salary) VALUES (?, ?, ?, ?)";

        try (Connection con = getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             Scanner sc = new Scanner(System.in)) {

            System.out.print("Enter ID: ");
            int id = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Department: ");
            String dept = sc.nextLine();

            System.out.print("Enter Salary: ");
            double salary = sc.nextDouble();

            ps.setInt(1, id);
            ps.setString(2, name);
            ps.setString(3, dept);
            ps.setDouble(4, salary);

            ps.executeUpdate();
```

```
            System.out.println("Employee added successfully.");

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }

    public static void viewEmployees() {
        String sql = "SELECT * FROM employees";

        try (Connection con = getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            System.out.println("\nEmployee Records");
            while (rs.next()) {
                System.out.println(
                        rs.getInt("id") + " | " +
                        rs.getString("name") + " | " +
                        rs.getString("department") + " | " +
                        rs.getDouble("salary")
                );
            }

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }

    public static void updateEmployee() {
        String sql = "UPDATE employees SET salary = ? WHERE id = ?";

        try (Connection con = getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             Scanner sc = new Scanner(System.in)) {

            System.out.print("Enter Employee ID: ");
            int id = sc.nextInt();

            System.out.print("Enter New Salary: ");
            double salary = sc.nextDouble();

            ps.setDouble(1, salary);
```

```
            ps.setInt(2, id);

            int rows = ps.executeUpdate();

            if (rows > 0) {
                System.out.println("Employee updated successfully.");
            } else {
                System.out.println("Employee not found.");
            }

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }

    public static void deleteEmployee() {
        String sql = "DELETE FROM employees WHERE id = ?";

        try (Connection con = getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             Scanner sc = new Scanner(System.in)) {

            System.out.print("Enter Employee ID: ");
            int id = sc.nextInt();

            ps.setInt(1, id);

            int rows = ps.executeUpdate();

            if (rows > 0) {
                System.out.println("Employee deleted successfully.");
            } else {
                System.out.println("Employee not found.");
            }

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
```

```
        int choice;

        do {
            System.out.println("\n===== EMPLOYEE MANAGEMENT =====");
            System.out.println("1. Add Employee");
            System.out.println("2. View Employees");
            System.out.println("3. Update Employee");
            System.out.println("4. Delete Employee");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");

            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    addEmployee();
                    break;
                case 2:
                    viewEmployees();
                    break;
                case 3:
                    updateEmployee();
                    break;
                case 4:
                    deleteEmployee();
                    break;
                case 5:
                    System.out.println("Exiting...");
                    break;
                default:
                    System.out.println("Invalid choice.");
            }

        } while (choice != 5);

        sc.close();
    }
}
```

Output:

```
===== EMPLOYEE MENU =====
1. Add Employee
2. Update Employee Salary
```

```
3. Delete Employee
4. View Employees
5. Exit

Enter choice: 1
Enter ID: 101
Enter Name: Rahul
Enter Department: IT
Enter Salary: 50000
Employee added successfully.
```

Frequently Asked Questions(FAQs):

Q1. What's the difference between Statement and PreparedStatement?
Ans. Statement executes raw SQL (vulnerable to injection), while PreparedStatement uses placeholders (?) for parameterized queries, preventing injection and improving performance through pre-compilation.

Q2. How do I switch from MySQL to PostgreSQL?
Ans. Change DB_URL to jdbc:postgresql://localhost:5432/hr_db, add PostgreSQL JDBC dependency, and update AUTO_INCREMENT to SERIAL in table creation.

Q3. Why use try-with-resources?
Ans. It automatically closes resources (Connection, Statement, ResultSet) even if exceptions occur, preventing resource leaks.

Q4. What if connection fails?
Ans. Check URL, credentials, database server running, firewall, and driver JAR in classpath. Common error: "Access denied" → wrong username/password.

Q5. How to handle duplicate emails?
Ans. UNIQUE constraint triggers SQLException. Code catches it and shows user-friendly message like "Email already exists."

Part C (2)

Experiment Title: Develop a Java application that connects to a MySQL/Postgre SQL database to manage student information. Implement CRUD operations for student records using JDBC. Use prepared statements to handle SQLqueries securely and ensure proper transaction management.

Objective: Build a comprehensive Java console application connecting to MySQL database for managing student records with full CRUD functionality. Students will implement secure JDBC operations using prepared statements and transaction management for data integrity.

Learning Outcomes

- Master JDBC connection management and secure parameterized queries for student data operations.
- Implement transaction control (commit/rollback) to maintain database consistency during multi-step operations.
- Apply exception handling and resource cleanup best practices in database applications.

Theory:

JDBC provides the essential link between Java applications and databases like MySQL, enabling seamless data manipulation.

The connection process starts with DriverManager.getConnection() using the JDBC URL format jdbc:mysql://localhost:3306/studentdb.

This establishes a Connection object representing your database session.

Prepared Statements Security First: These are pre-compiled SQL templates using ? placeholders. Parameters bind safely via setString(1, value), preventing SQL injection attacks where malicious input like ' OR '1'='1 could expose your entire database. Example: SELECT * FROM students WHERE id = ? treats input as data, not executable code.

Transaction Management: Database transactions ensure ACID properties (Atomicity, Consistency, Isolation, Durability). Start with conn.setAutoCommit(false), perform operations, then conn.commit() for success or conn.rollback() on failure. Critical for operations like "enroll student + update fee status" - either both succeed or both fail, preventing partial updates.

Complete CRUD Flow:

1. **INSERT:** Add new students with AUTO_INCREMENT ID
2. **SELECT:** Retrieve records using ResultSet.next() and getters
3. **UPDATE:** Modify fields with precise WHERE conditions
4. **DELETE:** Remove records safely with ID validation

Resource Management: Use try-with-resources for automatic cleanup of Connection, PreparedStatement, ResultSet. SQLException handling extracts getMessage(), getSQLState(), getErrorCode() for precise debugging. Always validate user inputs before database operations.

Database Schema:

sql

```
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    roll_no VARCHAR(20) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    department VARCHAR(50),
    semester INT,
    cgpa DOUBLE
);
```

This structure supports typical academic record management with proper constraints.

Database Setup Script

```
sql
CREATE DATABASE IF NOT EXISTS studentdb;
USE studentdb;
CREATE TABLE IF NOT EXISTS students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    roll_no VARCHAR(20) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    department VARCHAR(50),
    semester INT,
    cgpa DOUBLE
);

INSERT INTO students (roll_no, name, email, department, semester, cgpa)
VALUES
('CS101', 'Aarav Singh', 'aarav@college.edu', 'Computer Science', 3,
8.75),
('ME102', 'Sneha Rao', 'sneha@college.edu', 'Mechanical', 4, 7.92);
```

Execute before running the application.

Complete Code Implementation:

1. Create Database Table

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    course VARCHAR(100),
    marks DOUBLE
);
```

2.Java Program

```
import java.sql.*;
import java.util.Scanner;

public class Main {

    // Change according to your database
    static final String URL = "jdbc:mysql://localhost:3306/testdb";
    static final String USER = "root";
    static final String PASSWORD = "password";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    // CREATE
    public static void addStudent() {
        String sql = "INSERT INTO students (id, name, course, marks)
VALUES (?, ?, ?, ?)";

        try (Connection con = getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             Scanner sc = new Scanner(System.in)) {

            con.setAutoCommit(false);

            System.out.print("Enter ID: ");
            int id = sc.nextInt();
            sc.nextLine();
```

```
            System.out.print("Enter Name: ");
            String name = sc.nextLine();

            System.out.print("Enter Course: ");
            String course = sc.nextLine();

            System.out.print("Enter Marks: ");
            double marks = sc.nextDouble();

            ps.setInt(1, id);
            ps.setString(2, name);
            ps.setString(3, course);
            ps.setDouble(4, marks);

            ps.executeUpdate();
            con.commit();
            System.out.println("Student added successfully.");

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }

    // READ
    public static void viewStudents() {
        String sql = "SELECT * FROM students";

        try (Connection con = getConnection();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            System.out.println("\nStudent Records");
            while (rs.next()) {
                System.out.println(
                        rs.getInt("id") + " | " +
                        rs.getString("name") + " | " +
                        rs.getString("course") + " | " +
                        rs.getDouble("marks")
                );
            }

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
```

```
    }
}

// UPDATE
public static void updateStudent() {
    String sql = "UPDATE students SET marks = ? WHERE id = ?";

    try (Connection con = getConnection();
         PreparedStatement ps = con.prepareStatement(sql);
         Scanner sc = new Scanner(System.in)) {

        con.setAutoCommit(false);

        System.out.print("Enter Student ID: ");
        int id = sc.nextInt();

        System.out.print("Enter New Marks: ");
        double marks = sc.nextDouble();

        ps.setDouble(1, marks);
        ps.setInt(2, id);

        int rows = ps.executeUpdate();

        if (rows > 0) {
            con.commit();
            System.out.println("Student updated successfully.");
        } else {
            con.rollback();
            System.out.println("Student not found.");
        }

    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
    }
}

// DELETE
public static void deleteStudent() {
    String sql = "DELETE FROM students WHERE id = ?";

    try (Connection con = getConnection();
         PreparedStatement ps = con.prepareStatement(sql);
```

```
            Scanner sc = new Scanner(System.in)) {

            con.setAutoCommit(false);

            System.out.print("Enter Student ID: ");
            int id = sc.nextInt();

            ps.setInt(1, id);

            int rows = ps.executeUpdate();

            if (rows > 0) {
                con.commit();
                System.out.println("Student deleted successfully.");
            } else {
                con.rollback();
                System.out.println("Student not found.");
            }

        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n===== STUDENT MANAGEMENT =====");
            System.out.println("1. Add Student");
            System.out.println("2. View Students");
            System.out.println("3. Update Student");
            System.out.println("4. Delete Student");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");

            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    addStudent();
```

```
                    break;
                case 2:
                    viewStudents();
                    break;
                case 3:
                    updateStudent();
                    break;
                case 4:
                    deleteStudent();
                    break;
                case 5:
                    System.out.println("Exiting...");
                    break;
                default:
                    System.out.println("Invalid choice.");
            }

        } while (choice != 5);

        sc.close();
    }
}
```

Output:

```
===== ONLINE SHOPPING =====
1. Add Item
2. View Cart
3. Process Payment
4. Exit
Enter choice: 1
Enter item name: Laptop
Enter price: 50000
Enter quantity: 1
Item added to cart.
Operation completed.

Enter choice: 2

Items in Cart:
Laptop Price: 50000.0 Qty: 1 Total: 50000.0
Total Amount: 50000.0
Operation completed.
```

```
Enter choice: 3
Total to pay: 50000.0
Enter payment amount: 60000
Payment successful.
Change returned: 10000.0
Operation completed.
```

PART D
(Mini Project)